

Hospital Tech Challenge

Documentacao Tecnica do Backend

Arquitetura e Desenvolvimento Java - Fase 3

Spring Boot | Spring Security | GraphQL | RabbitMQ | PostgreSQL

Projeto academico para o Tech Challenge da FIAP, com foco em seguranca, arquitetura distribuida, comunicacao assincrona e consultas flexiveis via GraphQL.

Sumario

Este documento descreve a arquitetura e o funcionamento do projeto Hospital Tech Challenge, um backend hospitalar modular desenvolvido com Java e Spring Boot. A solucao tem como objetivo demonstrar, de forma pratica, os principais temas estudados na Fase 3: seguranca em aplicacoes Java, GraphQL, arquitetura distribuida, resiliencia e comunicacao assincrona com RabbitMQ.

O sistema foi organizado em dois servicos principais: appointment-service e notification-service. O appointment-service concentra as regras de autenticacao, autorizacao, agendamento de consultas, historico do paciente e publicacao de eventos. O notification-service consome os eventos publicados no RabbitMQ e simula o envio de lembretes aos pacientes.

A escolha por dois servicos busca representar uma arquitetura distribuida simples, adequada para o escopo academico, mas ainda alinhada a boas praticas de mercado. O servico de agendamento nao precisa conhecer os detalhes internos do servico de notificacoes; ele apenas publica uma mensagem no broker. Isso reduz acoplamento e permite que cada servico evolua separadamente.

A documentacao foi escrita para facilitar tanto a avaliacao do projeto quanto sua manutencao futura. Ela apresenta a visao funcional, a arquitetura, as tecnologias utilizadas, as regras de seguranca, os endpoints REST, as operacoes GraphQL, o fluxo assíncrono via RabbitMQ e as instrucoes de execucao e teste.

1. Contexto do desafio

O problema proposto envolve um ambiente hospitalar que precisa de um sistema para agendamento de consultas, gerenciamento do historico de pacientes e envio de lembretes automaticos. O sistema deve ser acessado por diferentes perfis de usuario, como medicos, enfermeiros e pacientes, cada um com permissoes especificas.

Nesse contexto, o backend precisa garantir que cada perfil acesse apenas as funcionalidades permitidas. Medicos podem visualizar e editar o historico de consultas. Enfermeiros podem registrar consultas e acessar historicos. Pacientes podem visualizar apenas suas proprias consultas.

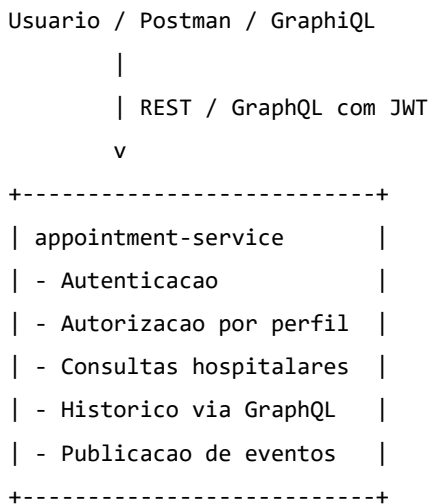
A solucao tambem precisa demonstrar comunicacao entre servicos. Quando uma consulta e criada ou alterada, o servico de agendamento publica um evento para que o servico de notificacoes processe a mensagem e envie um lembrete ao paciente. Esse fluxo usa mensageria para evitar dependencias diretas e para permitir processamento assincrono.

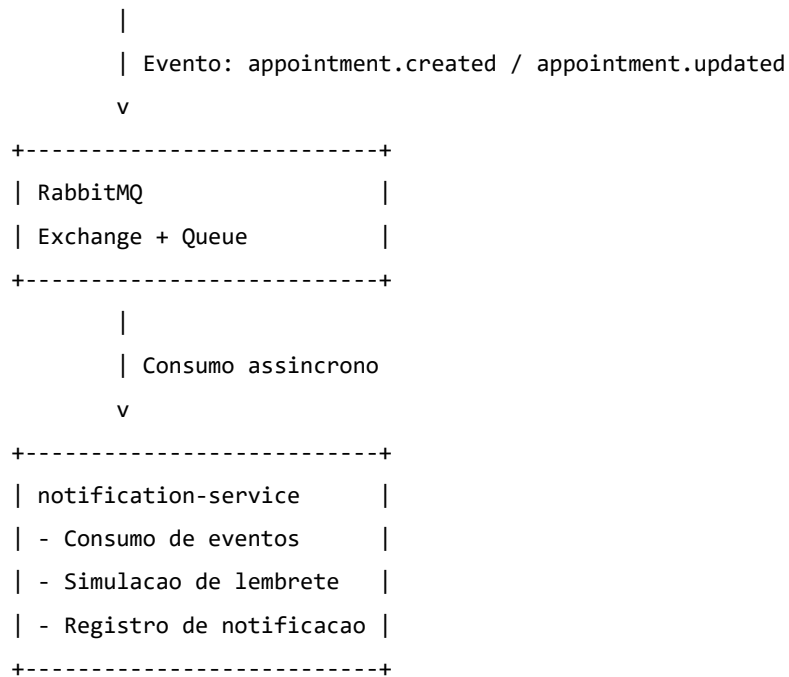
1.1 Objetivos da solucao

- Criar um backend hospitalar modular com Spring Boot.
- Aplicar Spring Security com autenticao e autorizacao por perfil.
- Expor consultas flexiveis de historico usando GraphQL.
- Separar responsabilidades em mais de um servico.
- Utilizar RabbitMQ para comunicacao assincrona entre servicos.
- Fornecer documentacao, collection de testes e instrucoes de execucao.

2. Visao geral da arquitetura

A arquitetura foi planejada para ser simples de executar em ambiente local, mas suficientemente clara para demonstrar os conceitos de sistemas distribuidos. A solucao e composta por dois microservicos, dois bancos PostgreSQL e um broker RabbitMQ.





Essa organizacao separa o fluxo transacional principal, que e o agendamento, do fluxo secundario, que e a notificacao. Caso o servico de notificacoes esteja temporariamente indisponivel, o servico de agendamento pode continuar funcionando, pois as mensagens ficam armazenadas no broker ate o consumo.

3. Servicos da aplicacao

3.1 appointment-service

O appointment-service e o servico principal da solucao. Ele possui as regras de negocio relacionadas a usuarios, pacientes, medicos e consultas. Tambem e responsavel por expor os endpoints REST, o endpoint GraphQL e a integracao com o RabbitMQ.

Responsabilidades principais:

- Autenticar usuarios e gerar token JWT.
- Controlar acesso com base em roles.
- Cadastrar, listar e atualizar consultas.
- Permitir consultas de historico por GraphQL.
- Publicar eventos de consulta criada e consulta atualizada.
- Persistir dados em PostgreSQL.

3.2 notification-service

O notification-service representa o modulo de notificacoes do sistema. Ele consome mensagens publicadas no RabbitMQ pelo appointment-service. Ao receber um evento, o servico simula o envio de um lembrete para o paciente e registra o processamento.

Responsabilidades principais:

- Conectar-se ao RabbitMQ como consumidor.
- Escutar a fila de notificacoes.
- Interpretar eventos de consulta criada ou atualizada.
- Gerar mensagem de lembrete para o paciente.
- Registrar ou logar a notificacao processada.

4. Tecnologias utilizadas

Tecnologia	Finalidade no projeto
Java 21	Linguagem principal do backend.
Spring Boot 3	Framework usado para criacao dos servicos.
Spring Security	Camada de autenticao e autorizacao.
JWT	Token utilizado para chamadas stateless.
BCrypt	Hash seguro para armazenamento de senhas.
Spring Data JPA	Abstracao para persistencia com banco relacional.
PostgreSQL	Banco de dados dos servicos.
Spring GraphQL	Implementacao de consultas flexiveis via GraphQL.

RabbitMQ	Broker de mensagens para comunicacao assincrona.
Docker Compose	Orquestracao local dos bancos e do broker.
Postman	Validacao dos endpoints REST.

A selecao das tecnologias foi feita considerando o conteudo estudado na fase e a facilidade de demonstracao em ambiente local. RabbitMQ foi escolhido por ser direto para cenarios de notificacao e por permitir demonstrar exchanges, filas, routing keys e consumers com baixa complexidade operacional.

5. Modelo de dominio

O dominio da aplicacao foi modelado com foco no problema hospitalar. O objetivo nao e cobrir todas as regras de um sistema hospitalar real, mas representar entidades suficientes para validar agendamento, historico e controle de acesso.

Entidade	Descricao
User	Representa um usuario autenticavel do sistema, contendo email, senha e perfil de acesso.
Patient	Representa o paciente que possui consultas e historico medico.
Doctor	Representa o medico responsavel por uma consulta.
Appointment	Representa uma consulta agendada, com paciente, medico, data, status e observacoes.
NotificationLog	Representa o registro de notificacao processada pelo notification-service.

5.1 Status da consulta

As consultas possuem status para indicar sua situacao atual dentro do fluxo de atendimento. Essa informacao pode ser usada em filtros GraphQL, exibicao para o paciente e historico do prontuario simplificado.

Status	Significado
SCHEDULED	Consulta agendada e ainda nao realizada.
COMPLETED	Consulta concluida.
CANCELED	Consulta cancelada.
RESCHEDULED	Consulta remarcada para nova data.

6. Seguranca da aplicacao

A seguranca foi estruturada com Spring Security. O fluxo de autenticacao utiliza email e senha. Quando as credenciais sao validas, a API retorna um token JWT. Esse token deve ser enviado nas proximas requisicoes no cabeçalho Authorization.

Authorization: Bearer <token_jwt>

As senhas dos usuarios sao armazenadas usando BCrypt, evitando armazenamento em texto puro. O controle de acesso e feito por perfis, com regras aplicadas aos endpoints REST e as operacoes sensiveis da aplicacao.

6.1 Perfis de acesso

Perfil	Permissoes principais
--------	-----------------------

MEDICO	Pode visualizar historico, listar consultas e editar consultas.
ENFERMEIRO	Pode registrar consultas e acessar historico.
PACIENTE	Pode visualizar apenas suas proprias consultas.

6.2 Justificativa do modelo de seguranca

O uso de JWT combina bem com APIs REST e microservicos porque evita a necessidade de sessao no servidor. Cada requisicao carrega suas credenciais de autorizacao no token. Isso facilita testes no Postman e mantem a aplicacao alinhada ao modelo stateless utilizado em muitos backends modernos.

7. Endpoints REST

Os endpoints REST concentram operacoes de comando, como login, criacao e atualizacao de consultas. As consultas mais flexiveis de historico ficam no GraphQL, separando bem os casos de uso.

Metodo	Endpoint	Acesso	Descricao
POST	/auth/login	Publico	Autentica usuario e retorna token JWT.
POST	/appointments	MEDICO, ENFERMEIRO	Cria uma nova consulta.
PUT	/appointments/{id}	MEDICO	Atualiza uma consulta existente.
GET	/appointments	MEDICO, ENFERMEIRO	Lista consultas cadastradas.
GET	/appointments/{id}	MEDICO, ENFERMEIRO, PACIENTE dono	Busca uma consulta especifica.

7.1 Exemplo de login

POST /auth/login

Content-Type: application/json

```
{
  "email": "medico@hospital.com",
  "password": "123456"
}
```

7.2 Exemplo de criacao de consulta

POST /appointments

Authorization: Bearer <token>

Content-Type: application/json

```
{
  "patientId": 1,
  "doctorId": 1,
  "dateTime": "2026-06-10T14:00:00",
  "notes": "Consulta de retorno"
}
```

Ao criar a consulta, alem de persistir o registro no banco, o appointment-service publica um evento no RabbitMQ. Esse evento sera processado pelo notification-service para simular o envio de lembrete ao paciente.

8. GraphQL

GraphQL foi utilizado para atender ao requisito de consultas flexíveis sobre o histórico médico. Diferente de endpoints REST fixos, o cliente pode especificar exatamente quais campos deseja retornar. Isso é útil para telas diferentes, como visão do médico, visão do enfermeiro e área do paciente.

8.1 Operações sugeridas

Query	Finalidade
appointmentsByPatient(patientId)	Lista todas as consultas de um paciente.
futureAppointmentsByPatient(patientId)	Lista apenas consultas futuras.
medicalHistory(patientId)	Retorna uma visão consolidada do histórico do paciente.

8.2 Exemplo de query: consultas por paciente

```
query {  
  appointmentsByPatient(patientId: 1) {  
    id  
    dateTime  
    status  
    notes  
    doctor {  
      name  
      specialty  
    }  
  }  
}
```

8.3 Exemplo de query: consultas futuras

```
query {  
  futureAppointmentsByPatient(patientId: 1) {  
    id  
    dateTime  
    status  
    doctor {  
      name  
    }  
  }  
}
```

9. Comunicacao assincrona com RabbitMQ

O RabbitMQ foi utilizado para desacoplar o servico de agendamento do servico de notificacoes. O appointment-service atua como producer, publicando eventos quando uma consulta e criada ou editada. O notification-service atua como consumer, processando esses eventos de forma independente.

9.1 Componentes de mensageria

Componente	Nome	Descricao
Exchange	hospital.appointments.exchange	Recebe mensagens do producer e roteia para as filas.
Queue	hospital.notifications.queue	Armazena eventos que serao consumidos pelo notification-service.
Routing key	appointment.created	Indica evento de consulta criada.
Routing key	appointment.updated	Indica evento de consulta atualizada.

9.2 Fluxo da mensagem

1. Medico ou enfermeiro cria uma consulta pelo appointment-service.
2. O appointment-service valida as permissoes do usuario.
3. A consulta e salva no PostgreSQL.
4. Um evento de dominio e publicado no RabbitMQ.
5. O notification-service consome a mensagem da fila.
6. O notification-service simula o envio de lembrete ao paciente.

9.3 Exemplo de payload do evento

```
{
  "appointmentId": 1,
  "patientId": 1,
  "patientEmail": "paciente@hospital.com",
  "doctorName": "Dr. Joao Silva",
  "appointmentDate": "2026-06-10T14:00:00",
  "eventType": "APPOINTMENT_CREATED"
}
```

Esse modelo e resiliente porque o servico de notificacoes pode ficar temporariamente indisponivel sem impedir o agendamento da consulta. Quando o consumer voltar, ele pode continuar processando as mensagens pendentes.

10. Persistencia e bancos de dados

A solucao usa PostgreSQL para persistir os dados dos servicos. Cada servico possui seu proprio contexto de persistencia, respeitando a ideia de separacao de responsabilidades em uma arquitetura distribuida.

Banco	Servico	Dados armazenados
appointment-db	appointment-service	Usuarios, pacientes, medicos e consultas.
notification-db	notification-service	Logs ou registros de notificacoes processadas.

Em ambientes reais, cada microservico deve ser dono de seus dados. Isso evita que um servico acesse diretamente as tabelas internas de outro servico e reduz o acoplamento entre os modulos.

11. Execucao do projeto

O projeto foi preparado para ser executado em ambiente local com Docker Compose e Maven. A ideia e facilitar a avaliacao e permitir que qualquer pessoa suba rapidamente os componentes necessarios, como RabbitMQ e PostgreSQL.

11.1 Pre-requisitos

- Java 21 instalado.
- Maven configurado no ambiente.
- Docker e Docker Compose instalados.
- Eclipse ou IntelliJ para importar os projetos Maven.
- Postman para executar a collection de testes.

11.2 Passos de execucao

1. Clonar o repositório do GitHub.
2. Executar docker compose up -d para subir RabbitMQ e bancos.
3. Importar appointment-service e notification-service no Eclipse como projetos Maven.
4. Executar o appointment-service.
5. Executar o notification-service.
6. Realizar login no Postman para obter token JWT.
7. Criar uma consulta e verificar se o evento foi processado pelo notification-service.

11.3 Comando principal

```
docker compose up -d
```

11.4 RabbitMQ Management

O RabbitMQ pode ser acompanhado pela interface de gerenciamento, normalmente disponivel em <http://localhost:15672>. Por essa interface e possivel verificar exchanges, filas, mensagens pendentes e consumidores conectados.

usuario e senha é

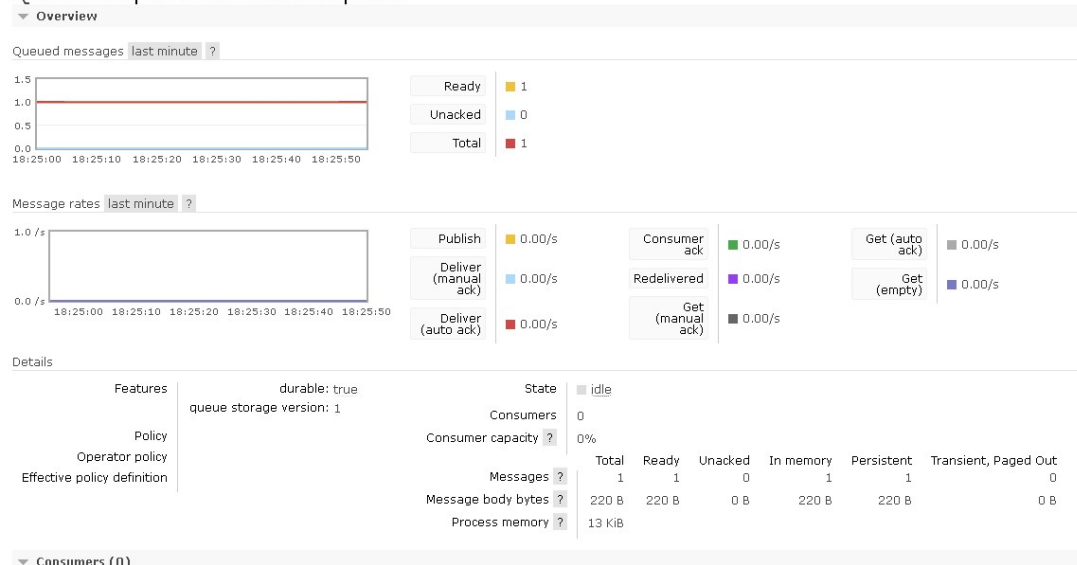
guest

guest

Um bom teste para validar se ta funcionando é

- Pare o notification-service no Eclipse.
- Crie uma consulta no Postman.
- Volte nessa tela do RabbitMQ.
- O campo Ready deve virar 1.
- Ligue o notification-service de novo.
- O Ready deve voltar para 0.

Queue hospital.notifications.queue



12. Criterios do desafio atendidos

Esta secao apresenta, de forma objetiva, como o projeto atende aos requisitos funcionais e tecnicos propostos no desafio. A ideia e facilitar a verificacao dos pontos exigidos na entrega e mostrar onde cada item foi contemplado na solucao.

Requisito	Situacao	Como foi implementado
Autenticacao com Spring Security	Atendido	Login com JWT, BCrypt e controle de usuarios.
Niveis de acesso por perfil	Atendido	Perfis MEDICO, ENFERMEIRO e PACIENTE.
Medicos visualizam/editam historico	Atendido	Permissao aplicada nas operacoes protegidas.
Enfermeiros registram consultas	Atendido	Endpoint de criacao liberado para enfermeiros.
Pacientes visualizam apenas suas consultas	Atendido	Regra de autorizacao restringe acesso ao proprio paciente.

GraphQL para historico	Atendido	Queries para consultas do paciente e consultas futuras.
Servico de agendamento	Atendido	appointment-service.
Servico de notificacoes	Atendido	notification-service.
Comunicacao assincrona	Atendido	RabbitMQ com exchange, queue e routing keys.
Collection de testes	Atendido	Collection Postman incluida no projeto.
Documentacao	Atendido	README e este documento tecnico.

A tabela acima pode ser usada durante a apresentacao para guiar a explicacao do projeto. Ela tambem ajuda o avaliador a encontrar rapidamente onde cada exigencia foi contemplada.

13. Plano de testes

A validacao do projeto deve ser feita por meio de testes manuais com Postman, GraphQL e observacao do RabbitMQ. O objetivo e demonstrar o fluxo completo, desde a autenticao ate o consumo da mensagem pelo servico de notificacoes.

13.1 Cenários principais

Cenario	Resultado esperado
Login com usuario medico	Deve retornar token JWT.
Login com credenciais invalidas	Deve retornar erro de autenticao.
Criar consulta como enfermeiro	Deve criar consulta e publicar evento.
Editar consulta como medico	Deve atualizar consulta e publicar evento.
Paciente consultar suas consultas	Deve retornar apenas dados do proprio paciente.
Paciente tentar acessar outro paciente	Deve negar acesso.
Executar query GraphQL de historico	Deve retornar consultas relacionadas ao paciente.
Parar notification-service e criar consulta	Mensagem deve permanecer no RabbitMQ ate o consumer voltar.
Subir notification-service novamente	Mensagem pendente deve ser consumida.

13.2 Evidencias recomendadas

- Print do login retornando token JWT.
- Print da criacao de consulta no Postman.
- Print da query GraphQL retornando historico.
- Print do RabbitMQ mostrando fila e consumer.
- Log do notification-service processando o lembrete.

14. Collection Postman

A collection Postman foi incluida para facilitar a avaliacao dos endpoints REST. O fluxo recomendado e executar primeiro o login, copiar o token retornado e usar esse token nas chamadas protegidas.

A collection deve conter, no minimo, chamadas para:

- Login de medico, enfermeiro e paciente.
- Criacao de consulta.
- Atualizacao de consulta.
- Listagem de consultas.

- Busca de consulta por ID.

Para GraphQL, os testes podem ser feitos pelo endpoint /graphql usando uma requisicao POST ou pela interface GraphQL, caso esteja habilitada no projeto.

15. Decisoes arquiteturais

15.1 Por que dois servicos?

Dois servicos sao suficientes para demonstrar separacao de responsabilidades sem tornar o projeto excessivamente complexo. O appointment-service cuida do dominio principal. O notification-service cuida de um processo secundario e assincrono. Essa divisao facilita a compreensao e atende ao requisito de separacao em mais de um servico.

15.2 Por que RabbitMQ?

RabbitMQ foi escolhido por ser adequado a cenarios de filas e notificacoes. Ele permite trabalhar com producer, consumer, exchange, queue e routing key de forma clara. Para o caso de uso de lembrete de consulta, a garantia de entrega e o desacoplamento entre servicos sao mais importantes do que processamento massivo de streaming.

15.3 Por que GraphQL apenas para consulta?

As operacoes de criacao e edicao foram mantidas em REST para simplificar o fluxo e facilitar testes com Postman. O GraphQL foi aplicado onde oferece maior valor: consultas flexiveis de historico, permitindo que o cliente escolha os campos necessarios.

15.4 Por que JWT?

JWT permite autenticacao stateless, facilitando a integracao entre cliente e API. O token carrega as informacoes necessarias para identificacao e autorizacao do usuario, reduzindo a necessidade de sessao no servidor.

16. Organizacao do repositorio GitHub

Para o repositorio ficar claro, a estrutura recomendada e manter os servicos separados em pastas e incluir documentacao e collection de testes na raiz.

```
hospital-tech-challenge/  
  appointment-service/  
  notification-service/  
  postman/  
    hospital-tech-challenge.postman_collection.json  
  docs/  
    documentacao-tech-challenge-fiap-revisada.pdf  
  docker-compose.yml  
  README.md
```

O README da raiz deve explicar rapidamente o objetivo do projeto, tecnologias, arquitetura, como executar, usuarios de teste, endpoints e fluxo de mensageria. Quanto mais direto o README estiver, mais facil sera para o avaliador rodar e entender a entrega.

17. Melhorias futuras

A solucao foi criada para atender ao escopo academico da Fase 3. Em uma evolucao futura, algumas melhorias poderiam ser adicionadas para aproximar o projeto de um ambiente produtivo real.

- Adicionar testes automatizados de unidade e integracao.
- Adicionar Spring Actuator com metricas de saude dos servicos.
- Criar dashboards com Prometheus e Grafana.
- Implementar Dead Letter Queue para mensagens com erro no RabbitMQ.
- Adicionar auditoria para registrar usuario que criou ou alterou consulta.
- Implementar refresh token para o fluxo JWT.
- Adicionar validacoes de conflito de horario para medicos e pacientes.
- Criar CI/CD com GitHub Actions para build e testes automatizados.
- Adicionar documentacao OpenAPI para os endpoints REST.

18. Conclusao

O projeto Hospital Tech Challenge demonstra uma solucao backend modular para um contexto hospitalar, aplicando os principais conceitos estudados na Fase 3. A aplicacao combina Spring Security, controle de acesso por perfis, GraphQL para consultas flexiveis, RabbitMQ para comunicacao assincrona e separacao em servicos independentes.

A arquitetura proposta reduz o acoplamento entre agendamento e notificacao, permitindo que o fluxo principal continue funcionando mesmo se o processamento de lembretes estiver temporariamente indisponivel. Essa decisao reforca conceitos de sistemas distribuidos, resiliencia e mensageria.